Co-Evolutionary Genetic Algorithms for Memorization-Constrained Robust Chess Opening Repertoires

Hazeera Muhammad Hashim, Minhaj ul Hassan, and Zainab Zahid
Department of Computer Science, Habib University
Karachi, Pakistan
{hm09247, mu08984, zz08969}@std.habib.edu.pk

***Abstract*—This paper studies whether a closure rule, optimized via Genetic Algorithm (GA) or local search techniques, improves the robustness of chess opening repertoires across varying opponent skill levels. The repertoire is memorization-constrained, retaining a budget of $\mathcal{B}$ moves per color, and must cover all opponent replies played in at least 15% of observed Lichess games at any reached position (closure-constrained). We compare five methodologies over 30 independent seeds: a most-played heuristic baseline, random search, a greedy hillclimber, a standard GA (STATIC), and a co-evolutionary GA (COEVOLVE) where the repertoire and opponent populations evolve together. A CVaR-weighted fitness objective was applied; held-out evaluation always measures the expected score across three rating bands (1000 – 1399, 1400 – 1799, 1800 – 2199) to reward both mean performance and worst-case robustness. The closure rule emerges as the dominant factor, yielding a consistent $+0.023$ improvement in held-out expected score with a perfect effect size ($A_{12} = 1.0$, $p < 7.5 \times 10^{-9}$), while COEVOLVE offers no statistically significant benefit over STATIC ($A_{12} = 0.45$, $p = 1.0$) despite a 35× computational cost. Furthermore, the greedy hillclimber matches GA-quality solutions at a fraction of the runtime, suggesting the closure-constrained fitness landscape responds well to local search.**

***Index Terms*—chess, co-evolutionary, constrained combinatorial optimization, genetic algorithm, opening repertoire**

I. INTRODUCTION

Chess opening preparation can be fundamentally modeled as an optimization problem under constraints. An opening repertoire is a compact set of preferred responses for both colors that a player selects to hold up against a range of opponents. Each move played has successive resulting moves that could be played by the opponent in response, requiring the player to anticipate a varied distribution of opponent behaviors. A player can at most memorize a limited capacity of moves to prepare beforehand. Hence, for players picking the right set of opening lines is a real and practical challenge that must perform well across skill levels rather than against just one type of player. The problem centers on the discrete, non-convex search space of the moves and their responses, and the finite memorization budget a player possesses. This makes opening preparation an instance of constrained combinatorial optimization across shifting opponent skill levels.

What makes this hard is the sheer scale of the search space. A database built from real Lichess games exposes hundreds of distinct positions. Picking lines that together cover all critical opponent replies, without wasting the memorization budget on marginal lines, involves a combinatorial search that is neither convex nor continuous. Simply relying on current computer-aided tools, such as Lichess Opening Explorer, SCID, and ChessBase, is insufficient as they do not jointly optimize a global repertoire objective. Using the lines with the highest frequencies or win rates ignores coverage gaps and the fact that different skill levels play very differently. Relying on these standard tools fails to guarantee the coverage of dangerous opponent replies or account for how opponent behavior shifts with skill level. Moreover, what makes a repertoire "good" is not fixed: lines that work well against a positional opponent might fail against an aggressive player, and a repertoire tuned to perform well on average can hide systematic weaknesses that overall statistics simply obscure.

We treat the problem as *robust constrained optimization*, defined formally as extracting a budget-constrained subgraph of the chess position graph where the system must handle the most likely opponent continuations. Repertoires are selected from millions of Lichess rapid and classical games organized by rating band, with per-band move statistics computed for each node. Fitness is optimized through a population of candidates exploring the space in parallel, with controlled evaluation conducted over 30 independent seeds. Comparisons are made using Wilcoxon signed-rank tests and Vargha-Delaney effect sizes, identifying the closure rule as a decisive factor yielding a perfect effect size ($A_{12} = 1.0, p < 7.5 \times 10^{-9}$) and a $+0.023$ improvement.

The central design question is what opponent model to train against. A standard GA (STATIC) evaluates against a fixed, uniform mix of all three rating bands. A co-evolutionary model (COEVOLVE) lets a population of opponent strategies adapt and evolve alongside the repertoire population, guided by performance, novelty, and a Hall of Fame of informative opponents. We ask which methodology yields optimal results: static evolution, adversarial co-evolution, random search, a most-played heuristic, or a greedy hillclimber.

We implemented the fully automated model and found that co-evolution adds no measurable benefit over the static GA ($p = 1.0$, $A_{12} = 0.45$) despite a 35× computational cost, while the greedy hillclimber matches GA-quality solutions, suggesting the closure-constrained landscape offers the best

accuracy-to-cost tradeoff via local search.

II. Related Work

A. Chess databases and opening book tools.

Walczak [1] showed that adapting an opening book to model opponent style produces measurable improvements, an early indication that opponent-awareness matters in opening preparation. More recently, Marzo and Servedio [2] analyzed large-scale Lichess data and found statistically significant differences in move preferences across rating bands, which directly motivates our per-band fitness formulation.

B. Self-learning evolutionary chess programs.

Fogel et al. [3] present a chess engine that evolves its evaluation parameters, material values, a positional value table, and neural networks, using GA through random mutation and competitive parent selection. Their player Blondie25 attained a performance rating of around 2550 under tournament conditions. Our paper adopts their hyperparameter heuristics, including the number of trials and generations.

C. Co-evolutionary algorithms.

Co-evolution refers to simultaneously evolving two interacting populations so that each drives improvement in the other. Hillis [4] demonstrated this in sorting networks, showing that co-evolving "parasite" programs prevents premature convergence. Rosin and Belew [5] formalized adversary informativeness and introduced Hall-of-Fame archives to prevent loss of historically best opponents. Both ideas inform our implementation. Ramos et al. [6] applied co-evolutionary methods to chess program development, providing a close precedent for the approach we take here.

D. Novelty and diversity.

Lehman and Stanley [7] showed that optimizing for behavioral novelty, rather than a fixed objective, can outperform conventional goal-directed search. We incorporate novelty as a secondary component of opponent fitness to prevent the opponent population from collapsing to a single adversarial mixture strategy.

E. Robust and risk-aware optimization.

CVaR (Conditional Value at Risk) is a standard measure for penalizing worst-case outcomes rather than just averaging over them [8]. We apply it across rating bands: the fitness function balances mean score with performance on the worst-performing band, which incentivizes robustness across skill levels rather than specialization toward one type of opponent.

III. Problem Formulation

A. Position Graph

We model chess as a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where each node $v \in \mathcal{V}$ is a chess position and each edge $(v, v') \in \mathcal{E}$ is a legal move from v to v'. Positions are stored as truncated FEN strings (piece layout, whose turn it is, castling rights, and en passant square). Each node records whose turn it is and its depth $d(v) \in \{0, \ldots, D_{\max}\}$, with $D_{\max} = 10$. For each rating band $b \in \mathcal{B} = \{1000 - 1399, 1400 - 1799, 1800 - 2199\}$, every edge stores how often that move was played and how the game ended (white win, draw, or loss), taken from the Lichess database.

B. Per-Band Move Policies

Players in different rating bands have different move preferences. For each band b and position v, we estimate the probability that a player in that band plays move m directly from game counts:

$$p_b(m \mid v) = \frac{N_b(v, m)}{\sum_{m'} N_b(v, m')}, \tag{1}$$

where $N_b(v, m)$ is the number of times move m was played from position v by players in band b.

C. Position Evaluation

To score a position without simulating a full game, we estimate the expected White win rate directly from observed outcomes. For position v in band b:

$$\hat{v}_b(v) = \frac{\text{wins}_b(v) + 0.5 \cdot \text{draws}_b(v)}{N_b(v)}, \tag{2}$$

where $N_b(v)$ is the total number of games passing through v in band b. A draw counts as half a win. Positions with no recorded games are assigned a score of 0.5.

D. Repertoire and Closure Constraint

A repertoire is not just a list of moves, it is a complete game plan. When it is our turn, we pick one move and commit to it. When it is the opponent's turn, we must have an answer ready for every reply they are likely to make.

We track two things: a set of committed moves (one per our-turn position), and a reached set — all positions the game can actually reach if both sides follow the repertoire. The budget B limits how many of our-turn moves we can commit to.

The closure rule fills in the gaps. At any opponent-turn position in the reached set, if the opponent plays a move in at least 15% of recorded games, the position after that move must also be in the reached set. In other words, the repertoire cannot ignore a common opponent reply just because it is inconvenient. Whenever a new position is added to the reached set, we immediately check all opponent replies at that position and add any that cross the 15% threshold, repeating this process until no new positions are added.

E. Scoring and Fitness

Once a repertoire is built, we score it by walking through the strategy tree position by position. At each position:

- If it is our turn and we have a committed move, we follow it and continue walking.
- If it is our turn but the position is a leaf (no committed move), we use the stored win-rate estimate for that position.

- If it is the opponent's turn, we take a weighted average of the scores of all their replies, weighted by how often players in that rating band choose each move.

This gives a single expected score for the repertoire against a given rating band. We do this separately for the white and black repertoires, then combine them into one number: the white repertoire's score plus the black repertoire's score averaged.

A good repertoire must hold up against all skill levels, not just one. Given an opponent mixture $\boldsymbol{\pi} \in \{\pi_1, \pi_2, \pi_3\}$ over the three rating bands, the fitness is:

$$F(\mathcal{R}) = \sum_{i=1}^{3} \pi_i \cdot s_{b_i}(\mathcal{R}) + \lambda \cdot \min_i(s_{b_i}(\mathcal{R})), \tag{3}$$

with $\lambda = 1.0$. The first term is the weighted average score across bands and the second adds a bonus equal to the score on the worst band. This pushes the optimizer to fix weak spots rather than pile up points against easier opponents.

IV. Methodology

A. Data Collection and Preprocessing

We crawl the Lichess Opening Explorer API [9] via a rate-limited breadth-first search from the starting position, collecting move-by-move play counts and outcomes for each of the three rating bands and for the aggregate population, down to $D_{\max} = 10$ ply (five full moves per side). Positions are included only when aggregate game counts exceed band-specific thresholds (5,000 at ply ≤ 4; 20,000 at ply ≤ 7; 50,000 at ply ≤ 10), and individual moves must appear in at least 5% of games at the parent position to be stored.

The training split covers all games before June 2025 and is used for policy computation, evaluation caching, and all fitness evaluations during optimization. The held-out split covers games from June 2025 onwards and is used only to evaluate the final best repertoire from each run, under a uniform opponent mixture.

B. System Architecture

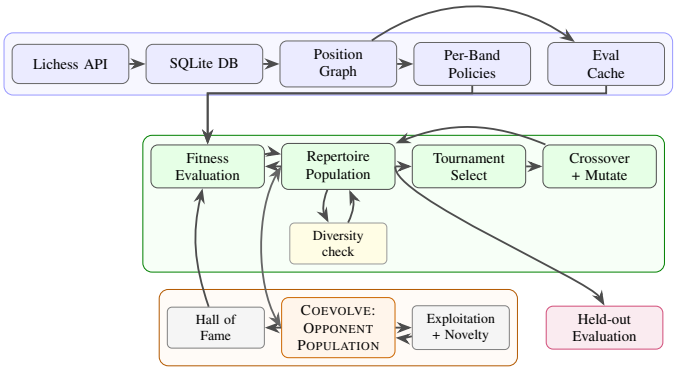

Fig. 1. System Architecture: Data Pipeline (blue), Genetic Algorithm Loop (green), Co-Evolutionary Opponent Population COEVOLVE only (orange), and Held-Out Evaluation (purple).

Fig. 1 shows the overall system architecture. The pipeline comprises of two phases: Data Preprocessing and GA Optimization.

C. Repertoire Chromosome and Initialization

Each individual in the GA population is a candidate: a paired white and black repertoire sharing the same position graph and subject to the same budget B. The initial population of $n = 50$ candidates are randomly initialized where at each our-turn node, we sample a move proportional to aggregate frequency.

D. Mutation Operators

Four mutation operators are applied with equal probability:

1) Move Swap: Selects a randomly committed node, replaces its chosen move with another legal alternative, and randomly rebuilds the subsequent subtree.
2) Extend: Targets a leaf node in the current repertoire tree, commits to a new move, and expands the resulting branch to satisfy the closure rule (provided sufficient budget remains).
3) Prune: Selects a randomly committed node and clears all commitments within its subtree, effectively reverting that branch to a leaf node to free up memorization budget.
4) Opening Replacement: Targets a committed node early in the game (ply ≤ 5), replacing its move and entirely reconstructing the downstream subtree via random initialization to encourage large-scale exploration.

Every mutation attempt enforces both the budget and closure constraints. If a constraint cannot be satisfied, the attempt fails and is retried and after 5 failed retries, the parent is returned unchanged.

E. Crossover

Crossover works by finding positions where both parents commit to the same move, these are shared pivot points. If any such pivots exist, one is sampled at random, and the offspring inherits parent A's decisions from the root down to the pivot and parent B's subtree below it. If the combined result would exceed the budget, the operation fails and parent A is returned unchanged. The same fallback applies when no shared pivot exists. The crossover rate is $p_c = 1.0$.

F. Selection and Diversity Maintenance

We use tournament selection with size $k = 2$. To prevent the population from collapsing to a single solution too quickly, we monitor mean pairwise Jaccard distance over the committed move sets after each generation. If this falls below $J_{\min} = 0.25$, meaning candidates have grown too similar to one another, 25% of the population is replaced with freshly random-initialized candidates.

G. Co-evolutionary Opponent Model

In COEVOLVE, the opponent model is dynamic rather than fixed. Each opponent is represented as a mixture $\boldsymbol{\pi} \in \Delta^2$ over the three rating bands, where Δ^2 denotes the 2-simplex. Intuitively, this encodes the weight assigned to each skill level when evaluating a repertoire. The COEVOLVE variant maintains a population of 50 such opponent chromosomes, each scored against the current repertoire population as follows:

$$f_{\text{opp}}(\boldsymbol{\pi}) = -\overline{F}(\boldsymbol{\pi}) + \eta \cdot d_{\text{nov}}(\boldsymbol{\pi}), \tag{4}$$

where $\overline{F}(\boldsymbol{\pi})$ is the mean repertoire fitness under mixture $\boldsymbol{\pi}$ (negated because a strong opponent is one that produces low scores), d_{nov} is the mean ℓ_2 distance to all other current opponents, and $\eta = 0.5$ is the novelty weight. Opponents reproduce via convex-blend crossover ($p_c = 1.0$) and mutation strength $s = 0.5$.

A practical challenge in co-evolution is that fitness scores become difficult to compare over time as the opponent population shifts. To ensure evaluation stability, we maintain a Hall of Fame: a fixed set of the 10 opponents that best distinguish strong repertoires from weak ones, identified by the highest fitness variance across the current repertoire population. Repertoires are always scored against these Hall-of-Fame opponents rather than the transient opponent pool, providing a consistent standard across generations.

H. Fitness Evaluation for Repertoires

In STATIC, every repertoire is evaluated against a fixed uniform mixture $\boldsymbol{\pi} = (1/3, 1/3, 1/3)$, treating all three rating bands equally throughout training. In COEVOLVE, each repertoire is instead scored against all current Hall-of-Fame opponents, and its fitness is the mean across them. Per-band scores $s_b(\mathcal{R})$ are cached on the candidate and reused when the same repertoire faces multiple opponents in the same generation, avoiding redundant computation.

I. Non-GA Baselines

To give context for what the GA actually does, we include three non-evolutionary baselines, each given the same evaluation budget of 10,000 fitness evaluations:

- Most-Played: A deterministic heuristic that performs a single greedy pass from the root, always committing to the move with the highest game frequency without further search.
- Random Search: A global search baseline that samples 10,000 independent candidates via random initialization and retains the individual with the highest fitness.
- Greedy Hillclimb: A local search approach that begins with the Most-Played solution and iteratively applies 10,000 random mutations, accepting a new candidate only if it strictly improves upon the current fitness.

J. Experimental Setup

Table I lists all fixed hyperparameters. All pairwise comparisons use a paired Wilcoxon signed-rank test with Holm correction; a result is significant when $p < 0.05$ after correction. The p-value gives the probability of observing the measured difference by chance under the null hypothesis of no effect. Effect size is reported as Vargha–Delaney A_{12}: 0.5 indicates no difference between methods, 1.0 means method A scores higher than method B on every seed, and 0.0 means the reverse.

TABLE I
FIXED HYPERPARAMETERS USED IN ALL EXPERIMENTS.

Parameter	Value
Memorization budget $\mathcal{B}$	25 / 10
Repertoire population n_R	50
Opponent population n_O	50
Generations G	100
Tournament size k	2
Crossover rate p_c	1.0
Mutation rate p_m	0.5
Closure threshold θ	0.15
CVaR weight λ	1.0
Novelty weight η	0.5
Hall of Fame size	10
Diversity threshold $J_{\min}$	0.25
Max ply depth $D_{\max}$	10
Seeds	30 (1000–1029)

The two budget levels correspond to qualitatively different repertoires suited to different player profiles. At $B = 10$, the best repertoires span 63 total positions across both colors, committing to 20 moves with lines typically extending 3–4 ply a compact, learnable set appropriate for players new to opening theory who want coherent first responses without deep memorization. At $B = 25$, the repertoire grows to 124 total positions with 45 committed moves, lines regularly reaching 5–6 ply into named systems such as the Caro-Kann and Queen's Gambit Declined. This depth suits players already familiar with opening principles who are building tournament-ready preparation. Testing both budgets lets us examine whether optimization methods scale consistently across the memorization demands of casual and experienced players.

V. RESULTS

A. Main Comparison

The three optimized methods Hillclimb, STATIC, and COEVOLVE converge to essentially the same held-out score, while the most-played baseline and random search fall well behind. At $B = 10$, all three reach 0.527 ± 0.002 against the baseline's 0.498, a gap of roughly 0.03 (Table II). Random search scores 0.522 ± 0.002, meaningfully lower than the other optimized methods despite the same evaluation budget. The pattern repeats at $B = 25$, where the optimized methods reach 0.525 ± 0.002 versus 0.492 for the baseline. One somewhat surprising result is that $B = 10$ slightly outperforms $B = 25$, suggesting that a tighter budget pushes the algorithm toward higher-value lines.

Fig. 2 and Fig. 3 show the full score distributions at $B = 10$ and $B = 25$, respectively. At both budget levels, the three

TABLE II
HELD-OUT SCORES FOR ALL FIVE METHODS AT BOTH BUDGET LEVELS ($n = 30$ SEEDS).

	Budget 10 (Primary)		**Budget 25**	
Method	*Unif. mean*	*Worst band*	*Unif. mean*	*Worst band*
Baseline	0.498 ± 0.000	0.493 ± 0.000	0.492 ± 0.000	0.488 ± 0.000
Random search	0.522 ± 0.002	0.515 ± 0.001	0.518 ± 0.002	0.513 ± 0.001
Hillclimb	0.527 ± 0.003	0.521 ± 0.002	0.525 ± 0.002	0.520 ± 0.001
STATIC	0.527 ± 0.002	0.521 ± 0.002	0.525 ± 0.002	0.520 ± 0.002
COEVOLVE	0.527 ± 0.002	0.521 ± 0.002	0.525 ± 0.002	0.520 ± 0.002

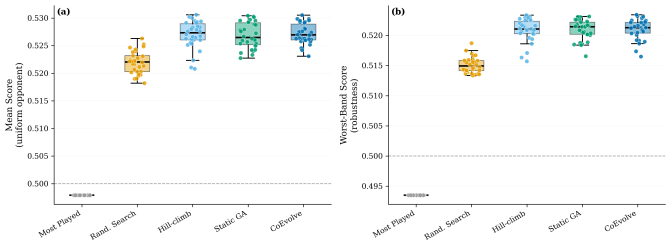

Fig. 2. Held-out score distributions at $B = 10$ ($n = 30$ seeds). All the optimized methods significantly outperform Random and Baseline.

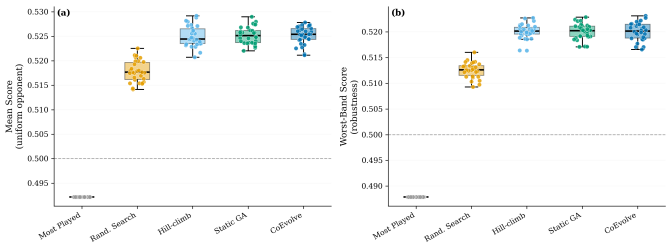

Fig. 3. Held-out score distributions at $B = 25$ ($n = 30$ seeds). All optimized methods significantly outperform Random and Baseline.

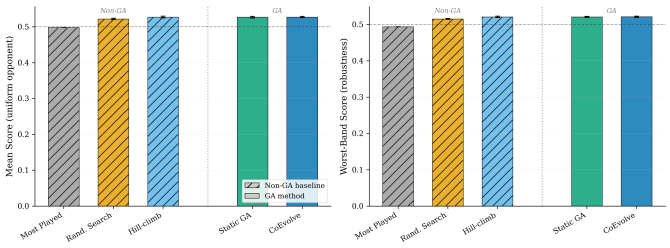

Fig. 4. GA vs. non-GA baselines at $B = 10$ (per-seed scores).

optimized methods are statistically indistinguishable from one another: Wilcoxon $p \geq 0.98$ and $A_{12} \in [0.45, 0.54]$ at $B = 10$ (Table III), and $p \geq 0.72$, $A_{12} \in [0.44, 0.48]$ at $B = 25$ (Table IV). All three significantly outperform random search ($p < 10^{-7}$, $A_{12} \leq 0.07$) and the most-played baseline ($p < 1.5 \times 10^{-8}$, $A_{12} = 0.0$) at both budgets, meaning no baseline seed beats any optimized seed.

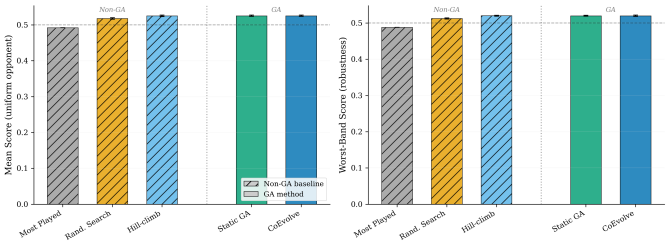

Fig. 5. GA vs. non-GA baselines at $B = 25$ (per-seed scores).

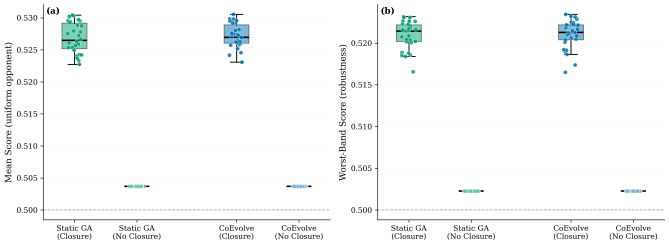

Fig. 6. Closure ON vs. OFF at $B = 10$.

B. GA vs. Non-GA Baselines

The key takeaway from Fig. 4 and Fig. 5 is that the greedy hillclimber matches both GA methods in final quality at both budget conditions, with tighter spread at $B = 10$. It is not a sophisticated algorithm, just steepest-ascent mutations from a greedy initialization, yet it finds solutions that neither GA variant can improve upon. Random search, which samples 10,000 candidates without structure, is significantly worse than all three, proving that focusing on small, local changes is a successful strategy for this task, whether you have a 10-move or a 25-move budget.

C. Closure Ablation

The closure rule turns out to be the most important component of the system. Removing it drops mean score by 0.023 at $B = 10$ and 0.0215 at $B = 25$, with $A_{12} = 1.0$ in all cases: every one of the 30 seeds performs worse without the rule, no exceptions (Table V, Fig. 6, Fig. 7). Worst-band score drops by 0.019 at $B = 10$ and 0.018 at $B = 25$, confirming that closure improves robustness, not just average performance, and that this benefit is consistent across both budget conditions. The statistical evidence is unambiguous: Wilcoxon $p < 7.5 \times 10^{-9}$

TABLE III
PAIRWISE STATISTICAL COMPARISONS BETWEEN ALL METHODS AT $B = 10$.

Method A vs. B	Δ	p-val	A_{12}	Sig.
Baseline vs. Random	-0.024	$< 10^{-8}$	0.00	✓
Baseline vs. Hillclimb	-0.029	$< 10^{-8}$	0.00	✓
Baseline vs. STATIC	-0.029	$< 10^{-8}$	0.00	✓
Baseline vs. COEVOLVE	-0.029	$< 10^{-8}$	0.00	✓
Random vs. Hillclimb	-0.005	$< 10^{-7}$	0.07	✓
Random vs. STATIC	-0.005	$< 10^{-8}$	0.06	✓
Random vs. COEVOLVE	-0.005	$< 10^{-8}$	0.03	✓
Hillclimb vs. STATIC	$+0.000$	1.00	0.54	–
Hillclimb vs. COEVOLVE	-0.000	0.98	0.51	–
STATIC vs. COEVOLVE	-0.000	1.00	0.45	–

TABLE IV
PAIRWISE STATISTICAL COMPARISONS BETWEEN ALL METHODS AT $B = 25$.

Method A vs. B	Δ	p-val	A_{12}	Sig.
Baseline vs. Random	-0.026	$< 10^{-8}$	0.00	✓
Baseline vs. Hillclimb	-0.033	$< 10^{-8}$	0.00	✓
Baseline vs. STATIC	-0.033	$< 10^{-8}$	0.00	✓
Baseline vs. COEVOLVE	-0.033	$< 10^{-8}$	0.00	✓
Random vs. Hillclimb	-0.007	$< 10^{-8}$	0.01	✓
Random vs. STATIC	-0.007	$< 10^{-8}$	0.00	✓
Random vs. COEVOLVE	-0.007	$< 10^{-8}$	0.00	✓
Hillclimb vs. STATIC	-0.000	0.90	0.45	–
Hillclimb vs. COEVOLVE	-0.000	1.00	0.44	–
STATIC vs. COEVOLVE	-0.000	0.72	0.48	–

TABLE V
EFFECT OF THE CLOSURE RULE ON HELD-OUT SCORES AT BOTH BUDGET LEVELS ($n = 30$ SEEDS).

Budget	Metric	Method	With	Without	Δ
$B = 10$	Unif. mean	STATIC	0.5269	0.5037	+0.0232
		COEVOLVE	0.5272	0.5037	+0.0235
	Worst band	STATIC	0.5209	0.5023	+0.0187
		COEVOLVE	0.5211	0.5023	+0.0188
$B = 25$	Unif. mean	STATIC	0.5252	0.5037	+0.0215
		COEVOLVE	0.5252	0.5037	+0.0215
	Worst band	STATIC	0.5200	0.5023	+0.0178
		COEVOLVE	0.5201	0.5023	+0.0178

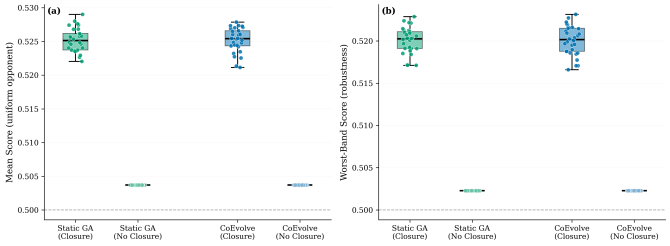

Fig. 7. Closure ON vs. OFF at $B = 25$.

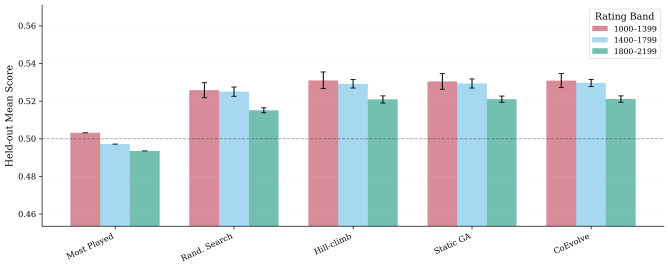

Fig. 8. Per-band held-out scores at $B = 10$.

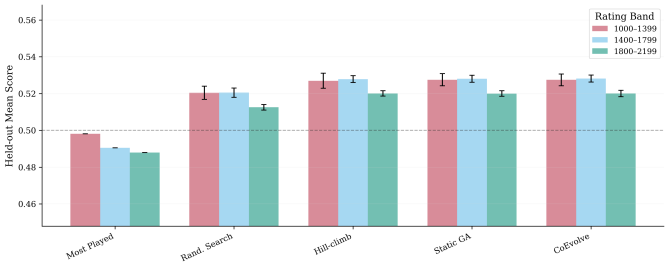

Fig. 9. Per-band held-out scores at $B = 25$.

with Holm correction. Notably, both STATIC and COEVOLVE converge to the same no-closure score of 0.5037, suggesting the unconstrained landscape has a dominant basin that the GA locates quickly regardless of variant.

D. Robustness Across Rating Bands

Fig. 8 and Fig. 9 reveal why a mean-only objective would be insufficient: the three rating bands show meaningfully different score profiles, with score gaps of up to 0.03 between bands. A repertoire that performs well against mid-range players does not automatically hold up against beginners or advanced opponents. The CVaR term in the fitness explicitly penalizes this variation. In practice, all optimized methods achieve comparable per-band robustness, with worst-band scores tracking the uniform mean closely (Table II).

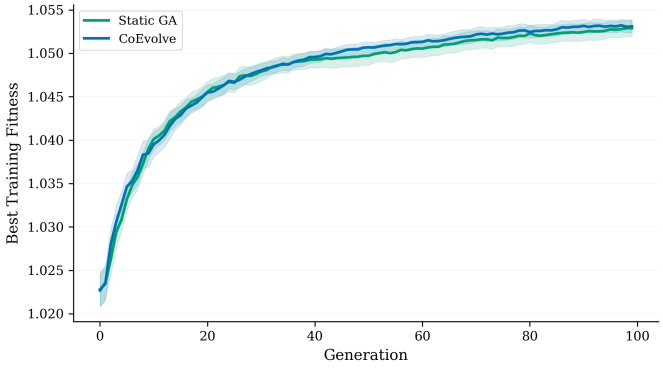

Fig. 10. Training fitness over 100 generations at $B = 10$ (mean $\pm$ 95% CI, $n = 30$ seeds).

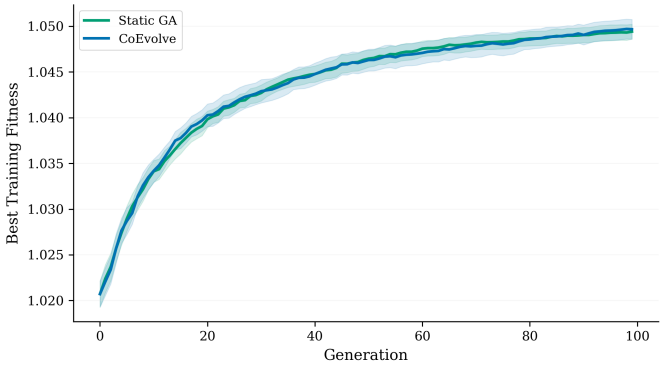

Fig. 11. Training fitness over 100 generations at $B = 25$ (mean $\pm$ 95% CI, $n = 30$ seeds).

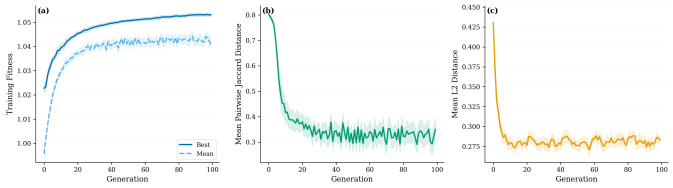

Fig. 12. CoEvolve dynamics at $B = 10$ ($n = 30$ seeds): training fitness (a), repertoire diversity (b), opponent diversity (c).

E. GA Convergence Dynamics

Fig. 10 and Fig. 11 show training fitness over 100 generations at $B = 10$ and $B = 25$, respectively. At both budget levels, Static and CoEvolve improve rapidly in the first 30 generations, then plateau. Their 95% confidence intervals overlap throughout the entire run, co-evolution provides neither a faster trajectory nor a higher ceiling at either budget. The diversity reinitialization mechanism keeps the population from converging prematurely; mean pairwise Jaccard distance settles at roughly 0.35 – 0.40 in later generations (visible in Fig. 12 and Fig. 13).

F. Coevolution Dynamics

Fig. 12 and Fig. 13 track three aspects of co-evolutionary behavior at $B = 10$ and $B = 25$. Repertoire diversity begins

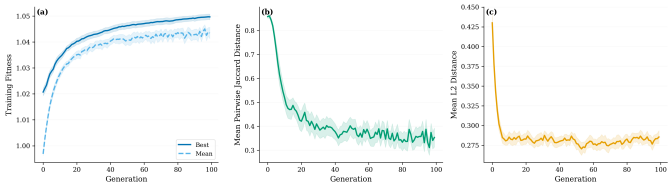

Fig. 13. CoEvolve dynamics at $B = 25$ ($n = 30$ seeds): training fitness (a), repertoire diversity (b), opponent diversity (c).

TABLE VI
Wall-clock runtime for all methods at both budget levels ($n = 30$ seeds).

Method	$B = 10$ **runtime (s)**	$B = 25$ **runtime (s)**
Baseline	0.03 ± 0.01	0.07 ± 0.01
Random search	2.85 ± 0.11	13.98 ± 5.35
Hillclimb	1.03 ± 0.06	5.16 ± 1.35
Static	2.67 ± 0.47	33.97 ± 129.1
CoEvolve	92.4 ± 11.6	371.6 ± 122.9

high (≈ 0.86 mean pairwise Jaccard) and settles around 0.35 by generation 25, with occasional spikes from diversity reinitialization, at both budget levels. Opponent diversity stabilizes quickly at ≈ 0.28 after about 10 generations, showing that the opponent population does maintain a slight spread of strategies. The Hall of Fame fills to its full capacity of 10 opponents by generation 0 and stays there, diverse, informative opponents are available from the very start. Yet despite all this adaptive machinery, the repertoire fitness trajectory is indistinguishable from Static at both budgets, which explains the null result in Table III and Table IV.

G. Example Repertoire

Fig. 14 shows a concrete example of what an optimized repertoire looks like at $B = 10$. The Sankey layout makes the effect of the closure rule immediately visible: at every opponent-turn node, the gray flows fan out to cover all replies above the 15% threshold, and a committed move (blue) is required for each. White commits to 1.d4 and follows with Queen's Gambit continuations (2.c4) against 1...d5 and the Jobava London System (2.Nc3) against 1...Nf6. Black meets 1.e4 with the Sicilian Defense (1...c5) and responds to 1.d4 with 1...Nf6, adopting King's Indian-style structures (g6, Bg7) or Benoni-style counters (c5) depending on White's continuation. The budget of 10 moves per color is nearly exhausted, confirming that the closure constraint forces the algorithm to make full use of the available budget to maintain coverage.

H. Computational Cost

The runtime differences between methods are stark (Table VI). CoEvolve takes 35$\times$ longer than Static at $B = 10$ and 11$\times$ longer at $B = 25$, yet produces no measurable improvement in held-out score. The greedy hillclimber is the clear efficiency winner, reaching Static-equivalent quality in just 1.0 s at $B = 10$. The high standard deviation for Static at $B = 25$ ($\pm$129.1 s) reflects variable tree sizes across seeds:

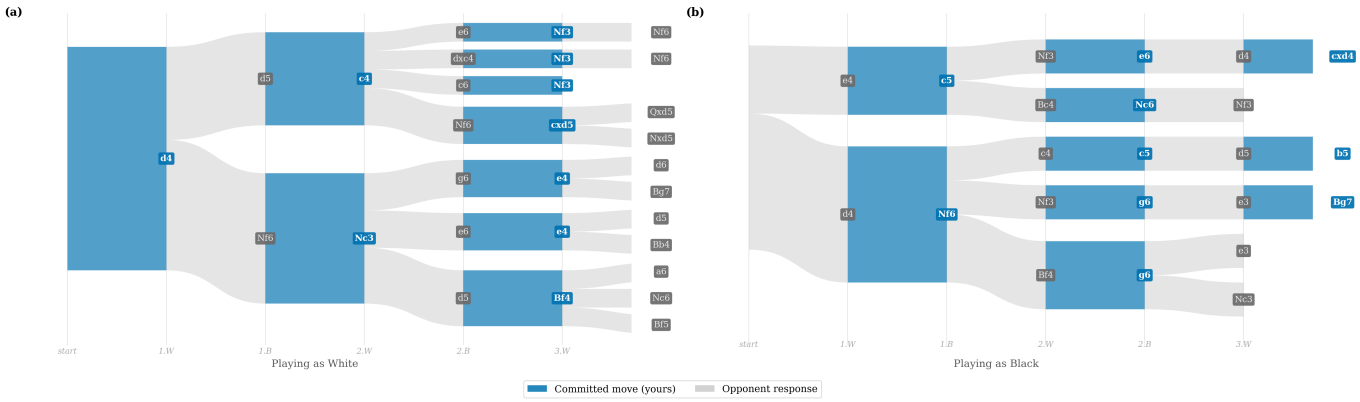

Fig. 14. Best COEVOLVE repertoire at $B = 10$, visualized as a Sankey diagram. Blue boxes represent committed moves; gray flows represent opponent responses. (a) As White, the model opens 1.d4, committing to the Queen's Gambit (2.c4) against 1...d5 and the Jobava London System (2.Nc3) against 1...Nf6. (b) As Black, the model meets 1.e4 with the Sicilian Defense (1...c5) and responds to 1.d4 with 1...Nf6, adopting King's Indian-style structures (g6, Bg7) or Benoni-style counters (c5) based on White's continuation.

some seeds encounter denser opening branches that require substantially more evaluation time.

VI. DISCUSSION

A. The Closure Rule Is the Decisive Contribution

The most important takeaway from this work is not the GA but the closure rule. Removing it drops held-out performance by 0.023 with perfect effect size ($A_{12} = 1.0$, $p < 7.5 \times 10^{-9}$), meaning every single seed across all 30 runs performs worse without it, across both GA variants and both budget conditions. The closure rule enforces what we call adversarial completeness: the repertoire must handle all likely opponent replies, not just the single most popular continuation. Without this constraint, the GA exploits the freedom to ignore inconvenient branches, producing repertoires that score well on training data but fail in practice as soon as an opponent plays a common alternative.

B. Co-Evolution Adds No Benefit Over a Static Opponent

COEVOLVE was designed with principled co-evolutionary machinery: adaptive opponent populations, novelty-driven fitness, and a Hall of Fame for evaluation stability. Yet it provides no statistically significant benefit over STATIC ($p = 1.0$, $A_{12} = 0.45$) at $35\times$ the computational cost. The most likely explanation is that the opponent search space, a 2-simplex over three rating bands, is simply too low-dimensional to generate meaningful adversarial pressure. The uniform mixture already captures most of the signal, leaving no useful gradient for co-evolution to exploit. A richer opponent model, such as per-position move policies that vary by playing style rather than just rating, might create enough adversarial diversity for co-evolution to pay off.

C. The Fitness Landscape Favours Simple Local Search

The greedy hill-climber matches both GA variants in final held-out quality at roughly $1/90$ of COEVOLVE's runtime at $B = 10$. This strongly suggests that the closure-constrained fitness landscape is largely unimodal, or at least contains a dominant basin of attraction that a simple local-search procedure can reach. At the scales tested here, maintaining a population of 50 candidates and running crossover brings no measurable benefit over a single trajectory of steepest-ascent mutations.

D. A Tighter Budget Does Not Hurt Performance

The smaller budget slightly outperforms the larger one (0.5272 vs. 0.5252 for COEVOLVE), which at first seems counter-intuitive. A tighter budget forces the algorithm to commit only to the highest-value moves, while a larger budget may fill remaining space with marginal lines that dilute overall quality. An alternative explanation is that $B = 25$ evaluations are individually more expensive, and 100 generations may not be sufficient for full convergence at that scale.

E. Limitations

The position graph covers only 10 ply, while real games regularly extend deeper. The opponent model aggregates all players within a rating band into a single mixture, ignoring individual style variation. The evaluation metric, an empirical-Bayes win rate from historical games, may not fully capture move value when an opponent knows they face a prepared repertoire. These gaps point directly to the most promising areas for future work.

VII. CONCLUSION

We presented a model for automatically constructing robust chess opening repertoires under a memorization budget, combining a genetic algorithm with a closure rule evaluated on real Lichess game data. The central finding is that the closure rule, which requires the repertoire to cover all opponent replies played in at least 15% of observed games, accounts for the bulk of the performance gain. It yields $+0.023$ in held-out expected score with Vargha–Delaney effect size ($A_{12} = 1.0$) across all 30 seeds and both budget conditions, confirming

that adversarial completeness is the essential ingredient for practical robustness.

The co-evolutionary component, despite its principled design, adds no measurable benefit over a static opponent ($p = 1.0$, $A_{12} = 0.45$), with the likely cause being that a 2-simplex opponent space is too restricted to generate exploitable adversarial pressure. More strikingly, a greedy hill-climber matches GA-level quality at $\sim$1/90 of COEVOLVE's runtime, indicating the closure-constrained landscape is well-suited to simple local search, and that population-based search offers diminishing returns at the scales tested.

The closure rule can be applied to any adversarial problem where preparation must be comprehensive. It serves as a necessary constraint in tasks where ignoring a likely path represents a total failure of the strategic model.

VIII. FUTURE WORK

A. Richer Opponent Models for Meaningful Co-Evolutionary Pressure

The 2-simplex opponent space is too constrained to expose meaningful adversarial diversity. We can replace the band weights with per-position move policies that capture individual playing style which would provide a far more expressive adversarial signal.

B. Deeper Position Graphs

Crawling to deeper ply would expose more opening systems, widen the performance gap between methods, and test whether closure remains the dominant factor in deeper trees.

C. Engine-Grounded Evaluation

Cross-referencing held-out win rates with Stockfish evaluations would separate "human exploits at this rating" from "is objectively stronger," clarifying the practical value of each repertoire.

D. Joint White–Black Co-Evolution

A single candidate co-evolving its White and Black repertoires simultaneously against a shared opponent population could discover synergistic opening choices across colors, rather than optimizing each side independently.

E. Real-Game Tournament Validation

All evaluation in this work is simulation-based, relying on historical win rates from the Lichess database. A natural extension is to test generated repertoires in live or correspondence play against human opponents. The two budget conditions map naturally to different player populations: $B = 10$ repertoires (20 moves, 3–4 ply) suit club-level players for whom memorization depth is a genuine constraint, while $B = 25$ repertoires (45 moves, 5–6 ply) are better evaluated against tournament players already comfortable with structured preparation. Real game outcomes would provide external validity that held-out win rates alone cannot, testing whether simulation-based gains translate to practice.

ACKNOWLEDGMENT

This work was completed as part of the Computational Intelligence (CS 462) course, Spring 2026, Habib University, Karachi, Pakistan. Opening data is sourced from the Lichess Opening Explorer API under the Creative Commons Attribution 4.0 license.

REFERENCES

[1] S. Walczak, "Improving opening book performance through modeling of chess opponents," in *Proc. ACM 24th Annu. Conf. Computer Science (CSC'96)*, Philadelphia, PA, USA, Feb. 1996, pp. 53–57.

[2] G. De Marzo and V. D. P. Servedio, "Quantifying the complexity and similarity of chess openings using online chess community data," *Scientific Reports*, vol. 13, no. 1, p. 5327, Apr. 2023.

[3] D. B. Fogel, "A self-learning evolutionary chess program," *Proceedings of the IEEE*, vol. 92, no. 12, pp. 1947–1964, Dec. 2004.

[4] W. D. Hillis, "Co-evolving parasites improve simulated evolution as an optimization procedure," *Physica D: Nonlinear Phenomena*, vol. 42, no. 1–3, pp. 228–234, 1990.

[5] C. D. Rosin and R. K. Belew, "New methods for competitive coevolution," *Evolutionary Computation*, vol. 5, no. 1, pp. 1–29, 1997.

[6] N. Ramos, S. Salgado, and A. C. Rosa, "Chess player by co-evolutionary algorithm," arXiv preprint arXiv:1605.06710, May 2016.

[7] J. Lehman and K. O. Stanley, "Abandoning objectives: Evolution through the search for novelty alone," *Evolutionary Computation*, vol. 19, no. 2, pp. 189–223, 2011.

[8] R. T. Rockafellar and S. Uryasev, "Optimization of conditional value-at-risk," *Journal of Risk*, vol. 2, no. 3, pp. 21–41, 2000.

[9] Lichess.org, "Lichess opening explorer API," https://lichess.org/api, 2025.